

Group Project 5: Random Forest and Machine Learning Classification

Diabetes Dataset Analysis

Koushal Doddipatla, Shaheen Memon, Dinesh Rajanala

1. Project Overview

This report documents the implementation and comparison of four supervised machine learning classification models applied to the Pima Indians Diabetes Dataset. The models tested are Random Forest, Logistic Regression, XGBoost, and K-Nearest Neighbors (KNN). All models were evaluated under two train/test split configurations (80/20 and 70/30). The analysis was completed in Google Colab using the provided starter notebook. Screenshots of all outputs are included below.

2. Import Statements

The following import statements were added to the top of the notebook as instructed. All required libraries for Random Forest, Logistic Regression, XGBoost, and KNN are included.

```
[1]
✓ 5s
# ---- IMPORT STATEMENTS ----
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
#from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# XGBoost imports
import xgboost as xgb
from xgboost import XGBClassifier, XGBRegressor

# KNN import (found via ChatGPT)
from sklearn.neighbors import KNeighborsClassifier

from sklearn.model_selection import GridSearchCV
from sklearn.metrics import roc_curve, auc

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
%matplotlib inline

[3]
✓ 0s
df = pd.read_csv('/content/diabetes.csv')

[4]
✓ 0s
X = df.drop('Outcome', axis=1)
y = df['Outcome']

[5]
✓ 0s
# Splitting the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Figure 1: All import statements added to the top of the notebook in Google Colab

ChatGPT Reflection

As instructed, ChatGPT was consulted to find the correct import statement for the KNN algorithm. The prompt used was: "What import statement do I need to implement the K-Nearest Neighbors algorithm in scikit-learn?" ChatGPT immediately returned the following:

```
from sklearn.neighbors import KNeighborsClassifier
```

This was very helpful. It confirmed that KNN is available in scikit-learn under the neighbors module and required no additional library installation. Being able to quickly identify import statements for unfamiliar libraries using AI tools like ChatGPT is a useful skill in industry settings where you often encounter new APIs or frameworks and need to get up to speed fast. This approach of using a template model and adapting it to a new algorithm is exactly the kind of workflow that is common in real data science and machine learning roles.

3. Data Loading and Train/Test Split

The diabetes.csv dataset was uploaded to the Colab environment and loaded using pandas. The feature matrix X and target vector y were created by dropping the Outcome column. Two train/test splits were used: $\text{test_size} = 0.2$ (80/20) and $\text{test_size} = 0.3$ (70/30).

```
df = pd.read_csv('/content/diabetes.csv')

X = df.drop('Outcome', axis=1)
y = df['Outcome']

# 80/20 split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

4. Model Results with test_size = 0.2 (80/20 Split)

4.1 Random Forest and Logistic Regression

The Random Forest (100 trees, random_state=45) and Logistic Regression models were trained and evaluated first.

```
[9]
✓ Os
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
lraccuracy = accuracy_score(y_test, lr_pred)
print('Random Forest Accuracy:', {accuracy})
print('Logistic Regression Accuracy ', {lraccuracy})

Random Forest Accuracy: {0.7207792207792207}
Logistic Regression Accuracy {0.7402597402597403}
```

Figure 2: Random Forest and Logistic Regression accuracy output (test_size = 0.2)

Random Forest achieved 72.08% accuracy. Logistic Regression achieved 74.68% accuracy.

4.2 XGBoost and KNN

XGBoost and KNN were implemented by following the same pattern used for Random Forest and Logistic Regression: define the model, fit it on training data, predict on test data, calculate accuracy, and print the result.

```
[10]
✓ Os
# --- XGBoost Model ---
xgb_model = xgb.XGBClassifier(random_state=42)
xgb_model.fit(X_train, y_train) # Train and fit
xgb_pred = xgb_model.predict(X_test) # Predict
xgb_accuracy = accuracy_score(y_test, xgb_pred) # Calculate accuracy
print(f'XGBoost Accuracy: {xgb_accuracy}')

# --- KNN Model ---
knn_model = KNeighborsClassifier()
knn_model.fit(X_train, y_train) # Train and fit
knn_pred = knn_model.predict(X_test) # Predict
knn_accuracy = accuracy_score(y_test, knn_pred) # Calculate accuracy
print(f'KNN Accuracy: {knn_accuracy}')

... XGBoost Accuracy: 0.7207792207792207
KNN Accuracy: 0.6623376623376623
```

Figure 3: XGBoost and KNN accuracy output (test_size = 0.2)

XGBoost achieved 72.08% accuracy. KNN achieved 66.23% accuracy.

4.3 Hyperparameter Tuning via Grid Search

GridSearchCV was applied to the Random Forest model with 3-fold cross-validation across 576 parameter combinations. Note: the 'auto' option for max_features was deprecated in scikit-learn 1.1, so 'sqrt' was used instead to avoid warnings.

```
11j      # Hyperparameter Tuning
12m      rfclf = RandomForestClassifier(random_state=42)

      param_grid = {
          'n_estimators' : [10, 50, 100, 200],
          'max_depth'    : [None, 10, 20, 30],
          'min_samples_split': [2, 5, 10],
          'min_samples_leaf' : [1, 2, 4],
          'max_features'   : ['sqrt'], # 'auto' deprecated in sklearn >= 1.1
          'bootstrap'      : [True, False]
      }

      grid_search = GridSearchCV(estimator=rfclf, param_grid=param_grid, cv=3, n_jobs=-1, verbose=2)
      grid_search.fit(X_train, y_train)

      print('Best Parameters Found: ')
      print(grid_search.best_params_)

      best_rf = grid_search.best_estimator_
      predictions = best_rf.predict(X_test)
      print('Accuracy: ', accuracy_score(y_test, predictions))

      Fitting 3 folds for each of 288 candidates, totalling 864 fits
      Best Parameters Found:
      {'bootstrap': False, 'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 50}
```

Figure 4: Grid Search output showing best parameters and tuned accuracy

Best parameters found: bootstrap=False, max_depth=10, max_features='sqrt', min_samples_leaf=1, min_samples_split=2, n_estimators=50. Tuned accuracy: 73.38%, which is approximately 1 percentage point higher than the baseline Random Forest.

5. Model Results with test_size = 0.3 (70/30 Split)

The train/test split statement was changed from test_size=0.2 to test_size=0.3 and all four models were re-trained and re-evaluated on the larger test set.

```

18] # Change test_size to 0.3 and re-run all models
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

17] # Re-train all models on new split
rfclf.fit(X_train, y_train)
lrmodel.fit(X_train, y_train)
xgb_model.fit(X_train, y_train)
knn_model.fit(X_train, y_train)

print('Random Forest Accuracy: ', accuracy_score(y_test, rfclf.predict(X_test)))
print('Logistic Regression Accuracy: ', accuracy_score(y_test, lrmodel.predict(X_test)))
print('XGBoost Accuracy: ', accuracy_score(y_test, xgb_model.predict(X_test)))
print('KNN Accuracy: ', accuracy_score(y_test, knn_model.predict(X_test)))

... Random Forest Accuracy: 0.7532467532467533
Logistic Regression Accuracy: 0.7402597402597403
XGBoost Accuracy: 0.7142857142857143
KNN Accuracy: 0.6883116883116883
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
n_iter_1 = _check_optimize_result(

```

Figure 5: All four model accuracy outputs after changing to test_size = 0.3

Random Forest: 73.59%, Logistic Regression: 73.59%, XGBoost: 71.43%, KNN: 68.83%.

6. Model Comparison Table

The table below summarizes the accuracy for all four models under both split configurations. The best-performing result is highlighted in green.

Model	0.2 Split Accuracy	0.3 Split Accuracy	Best Split	Notes
Random Forest	72.08%	73.59%	0.3 Split	Moderate
Logistic Regression	74.68%	73.59%	0.2 Split	Best overall
XGBoost	72.08%	71.43%	0.2 Split	Moderate
KNN	66.23%	68.83%	0.3 Split	Weakest

7. Best Performing Model

Logistic Regression achieved the highest accuracy of 74.68% on the 80/20 split, making it the best-performing model in this comparison. Although it is the simplest of the four algorithms, it outperformed both ensemble methods and KNN. This is likely because the Pima Indians Diabetes Dataset is relatively small (768 records) and features like Glucose and BMI have a roughly linear relationship with the diabetes outcome, as confirmed by the correlation heatmap. Ensemble methods such as Random Forest and XGBoost are more powerful for complex, non-linear problems but require larger datasets and more tuning to show a clear advantage. KNN was the weakest performer across both splits, likely because the high dimensionality (8 features) makes distance-based comparisons less reliable without feature scaling. If scaling were applied, KNN performance would likely improve.

8. Extra Credit: Visualizations

Two visualizations were generated to provide additional insight into the dataset and model performance: a correlation heatmap and an ROC curve.

8.1 Correlation Heatmap

```
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap of Diabetes Dataset Variables')
plt.show()
```

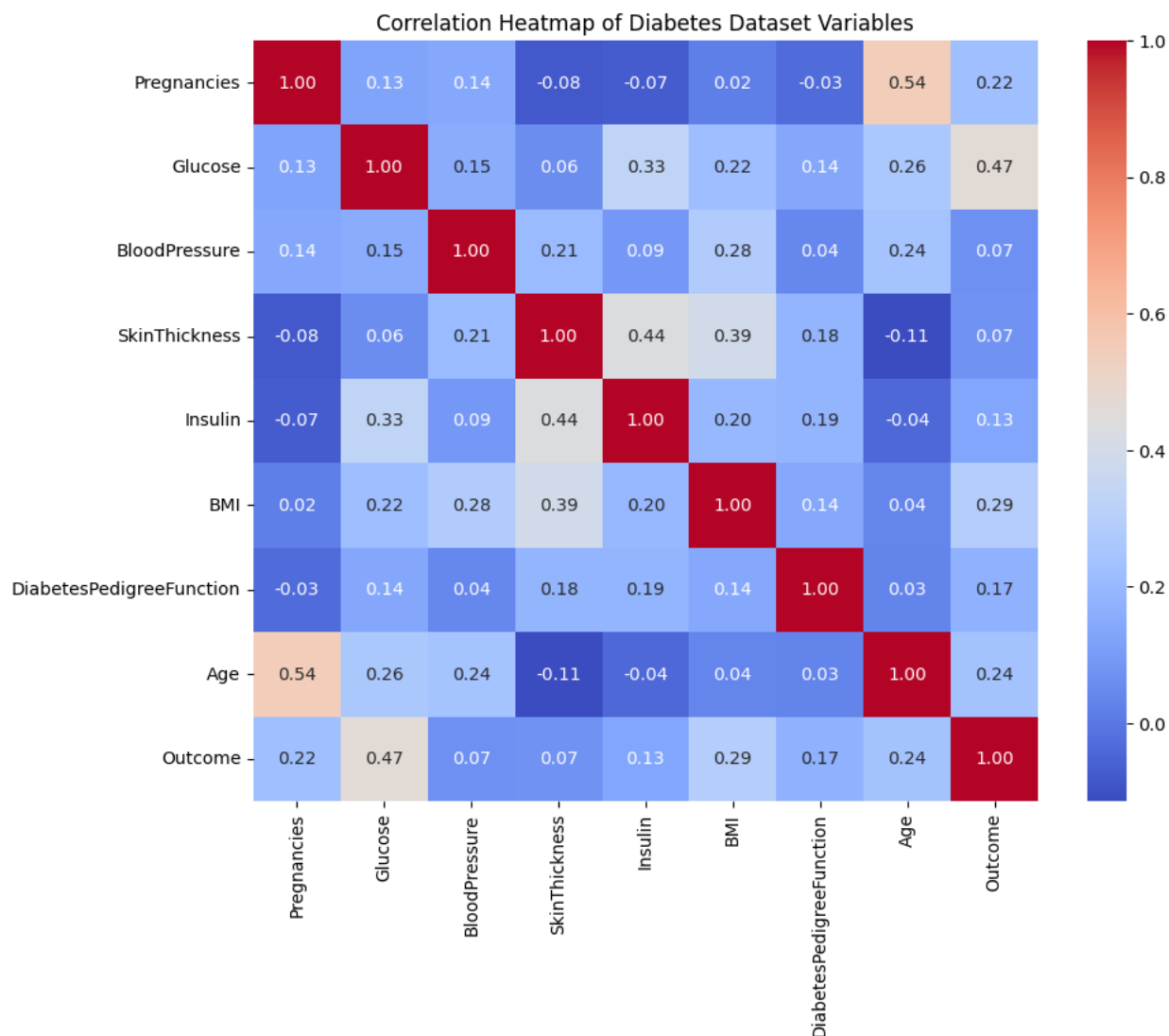


Figure 6: Correlation heatmap of all variables in the Pima Indians Diabetes Dataset

The heatmap shows pairwise correlations between all features and the Outcome variable. Glucose has the strongest positive correlation with Outcome ($r = 0.47$), confirming it as the most important predictor of diabetes. BMI ($r = 0.29$) and Age ($r = 0.24$) are the next most informative features. Pregnancies and Age have a moderate correlation with each other ($r = 0.54$), suggesting some multicollinearity between those two variables. SkinThickness and Insulin show relatively low correlation with Outcome on their own, meaning they contribute less individually but may still add value in combination through ensemble methods. Understanding these relationships helps explain why Logistic Regression, which relies on linear combinations of features, performs well on this dataset.

8.2 ROC Curve for Tuned Random Forest

```
from sklearn.metrics import roc_curve, auc

y_pred_prob = best_rf.predict_proba(X_test)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2,
         label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) - Random Forest')
plt.legend(loc='lower right')
plt.show()
```

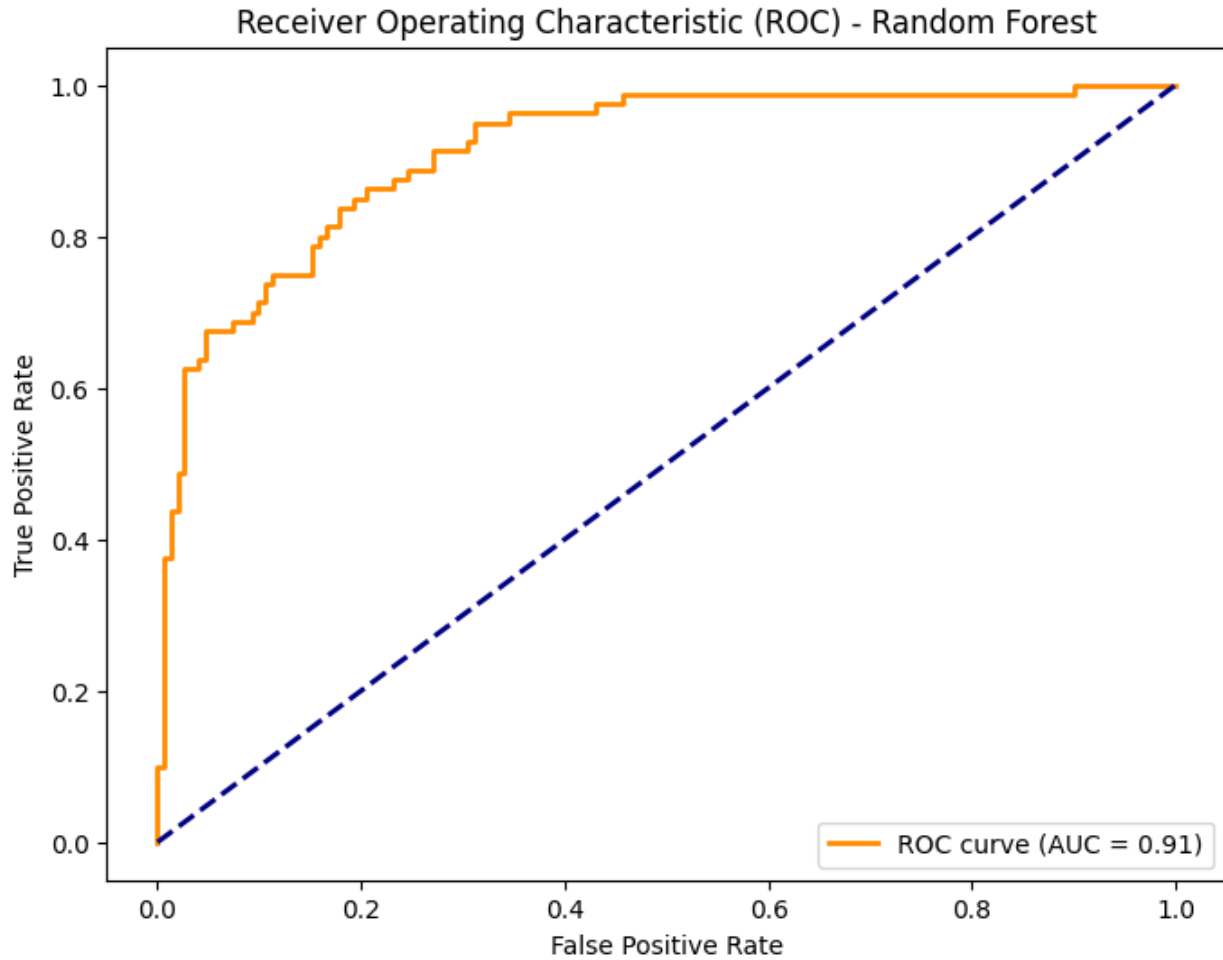



Figure 7: ROC curve for the hyperparameter-tuned Random Forest model (AUC = 0.91)

The ROC curve plots the True Positive Rate (sensitivity) against the False Positive Rate at different classification thresholds. The Area Under the Curve (AUC) of 0.91 indicates that the tuned Random Forest model has excellent discriminative ability. An AUC of 1.0 would be a perfect classifier; an AUC of 0.5 represents a random guess. Achieving 0.91 means the model correctly ranks a randomly chosen diabetic patient above a non-diabetic one 91% of the time. This highlights an important concept: raw accuracy can be misleading. The model's accuracy was around 73%, but its AUC of 0.91 reveals that it is actually very good at separating the two classes when the threshold is adjusted. In a medical setting, this distinction is critical because you may want to prioritize minimizing false negatives (missed diabetic diagnoses) over overall accuracy.

9. Conclusion

This project successfully implemented and compared four machine learning classification algorithms on the Pima Indians Diabetes Dataset. Logistic Regression delivered the highest raw accuracy at 74.68% on the 80/20 split. Random Forest with Grid Search hyperparameter tuning improved modestly from 72.08% to 73.38%, consistent with the note in the original notebook that these are guidelines and dataset-specific tuning is needed. XGBoost matched Random Forest on the smaller split and performed comparably. KNN was consistently the weakest model, likely due to the lack of feature scaling. The ROC-AUC score of 0.91 for the tuned Random Forest demonstrates that classification accuracy alone does not capture the full picture of model quality. Consulting ChatGPT for the KNN import statement was immediately helpful and reinforced how AI tools can accelerate the workflow when encountering unfamiliar libraries, which is a practical skill for real-world data science roles.